Chandigarh University

University Institute of Computing (UIC)

CASE STUDY

Polymorphism in C++

A case study on compile-time and run-time polymorphism through a payroll system

Name	Khushi
UID	25BCD10133
Course	BCA Data Science, 2nd Year
Subject	C++ Object-Oriented Programming (25CAH-201)
Submitted to	Mr. Jitendra Kumar
Academic session	2026-2027

1. Introduction

The word *polymorphism* comes from the Greek for "many forms". In C++ it means that one name, one function call or one operator can behave differently depending on the type of data or object it is used with. It is one of the four pillars of object-oriented programming, together with encapsulation, inheritance and abstraction.

Polymorphism is easy to define but harder to appreciate from a definition alone. Its real value appears when a program has to deal with many related kinds of objects and is expected to grow. This case study therefore looks at polymorphism through one concrete situation: a small company payroll program that must handle several kinds of employees and must keep working when a new kind is added.

The study first shows the problem when polymorphism is not used, then the theory behind the two kinds of polymorphism in C++, then the solution using virtual functions, and finally a comparison of the two designs. Every program in this report was compiled with `g++ -std=c++17 -Wall` and run, and the outputs shown are the actual outputs.

2. Background of the Scenario

The scenario is a fictional company, **TechNova**, that wants a program to calculate the monthly pay of its staff. The scenario is invented for this study, but the situation is typical of real software. At the start the company has three kinds of employees, and each is paid by a different rule.

Employee type	Pay rule	Example
Manager	Base salary plus a fixed allowance of 20,000	Asha, base 60,000, pay 80,000
Developer	Base salary plus a 10 percent bonus	Ravi, base 50,000, pay 55,000
Intern	Stipend only, no extras	Meera, base 15,000, pay 15,000

Some months later the company starts hiring **contractors**, who are paid by the hour (rate multiplied by hours worked). The payroll program has to be extended to handle them. This second step is what makes the design important: a good design absorbs the change easily, while a poor one needs edits in many places.

3. Problem Statement

The company needs a payroll program that:

1. calculates the pay of every employee according to the rule for that employee's type;
2. prints a payslip line for each employee and the total payroll;
3. can accept a new employee type without rewriting code that already works.

3.1 First attempt: a design without polymorphism

A common first solution is to store every employee in one structure, mark the type with an `enum`, and write one function that decides the pay with an if-else chain.

```
enum Type { MANAGER, DEVELOPER, INTERN };

struct Employee {
    string name;
    Type type;
```

```
    double base;
};

double calculatePay(const Employee& e) {
    if (e.type == MANAGER)
        return e.base + 20000;
    else if (e.type == DEVELOPER)
        return e.base + 0.10 * e.base;
    else if (e.type == INTERN)
        return e.base;
    return 0;
}
```

Running this program for Asha, Ravi and Meera gives the correct result:

```
Asha : 80000
Ravi : 55000
Meera : 15000
```

The program works, so the weakness is not visible yet. It appears when the program grows.

3.2 What goes wrong as the program grows

- **Every change touches old code.** To add contractors, the programmer must add a new value to the `enum`, add another branch to `calculatePay()`, and also add a field for hours to the shared structure that every other employee type has to carry unused.
- **The same type check is repeated.** If the company later adds a function for tax, for leave or for a payslip title, each function needs its own if-else chain on the type. Forgetting one branch in one function is a silent bug, because the compiler does not warn about it.
- **Data and behaviour are separated.** The rule for a manager lives far from anything that describes a manager, so the code is harder to read and to test.

These three problems are exactly the ones that polymorphism is designed to remove.

4. Theory: Polymorphism in C++

C++ supports two kinds of polymorphism, which differ in *when* the compiler decides which code to run.

Feature	Compile-time (static) polymorphism	Run-time (dynamic) polymorphism
Decided	During compilation	While the program is running
Also called	Early binding	Late binding
Achieved through	Function overloading, operator overloading, templates	Inheritance with virtual functions
Speed	Fastest; no extra lookup	Small cost for one extra lookup per call
Flexibility	Types must be known when compiling	New types can be added without changing existing code

4.1 Compile-time polymorphism

Here the compiler looks at the arguments or types in a call and picks the matching version before the program runs. There are three forms.

Function overloading means several functions share one name but differ in their parameters. **Operator overloading** gives an operator such as `+` a meaning for a user-defined class. **Templates** let one function or class work for many data types, with the compiler generating the right version for each type used.

```
int area(int side)         { return side * side; }
int area(int l, int b)     { return l * b; }
double area(double radius) { return 3.14159 * radius * radius; }

class Money {
    int rupees;
public:
    Money(int r = 0) : rupees(r) {}
    Money operator+(const Money& o) const { return Money(rupees + o.rupees); }
    void show() const { cout << "Rs. " << rupees << endl; }
};

template <typename T>
T largest(T a, T b) { return (a > b) ? a : b; }
```

Calling `area(5)`, `area(4, 6)` and `area(2.5)`, adding two `Money` objects of 500 and 250, and calling `largest(10, 20)` and `largest(3.5, 1.2)` produced:

```
25
24
19.6349
Rs. 750
20
3.5
```

Each call chose its version at compile time: the single `int` gave a square, the two integers a rectangle, the `double` a circle, and the template produced one version for `int` and another for `double`.

4.2 Run-time polymorphism

Run-time polymorphism is the kind that solves the payroll problem. It rests on three ingredients working together.

1. **A base class pointer or reference** that can refer to an object of any derived class.
2. **A virtual function** declared in the base class and **overridden** in each derived class.
3. **A call through the pointer or reference**, so the version that runs depends on the real object, not on the pointer's declared type.

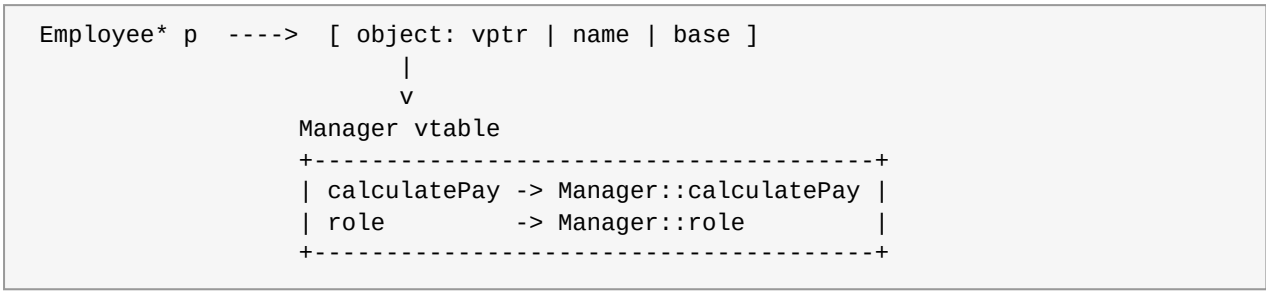
Without the keyword `virtual`, C++ picks the function from the pointer's type, which is the base class. With `virtual`, the choice is delayed until the call is made, and the object's actual type decides.

4.3 How virtual functions work internally

C++ implements virtual calls with two hidden structures.

- **The vtable (virtual table).** The compiler builds one table for each class that has virtual functions. It is an array of addresses, one for each virtual function, holding the address of the version that applies to that class.
- **The vptr (virtual pointer).** Every object of such a class carries one hidden pointer, set by the constructor, that points to its class's vtable.

When the program executes `staff[i]->calculatePay()`, it follows the object's vptr to the vtable, picks the entry for `calculatePay`, and jumps to that address. This is why the right version runs even though the pointer's type is the base class. The cost is one extra memory lookup per call and one extra pointer per object, which is small for most programs.



The same pointer type `Employee*` pointing to a `Developer` object would follow that object's vptr to the Developer vtable and reach `Developer::calculatePay` instead.

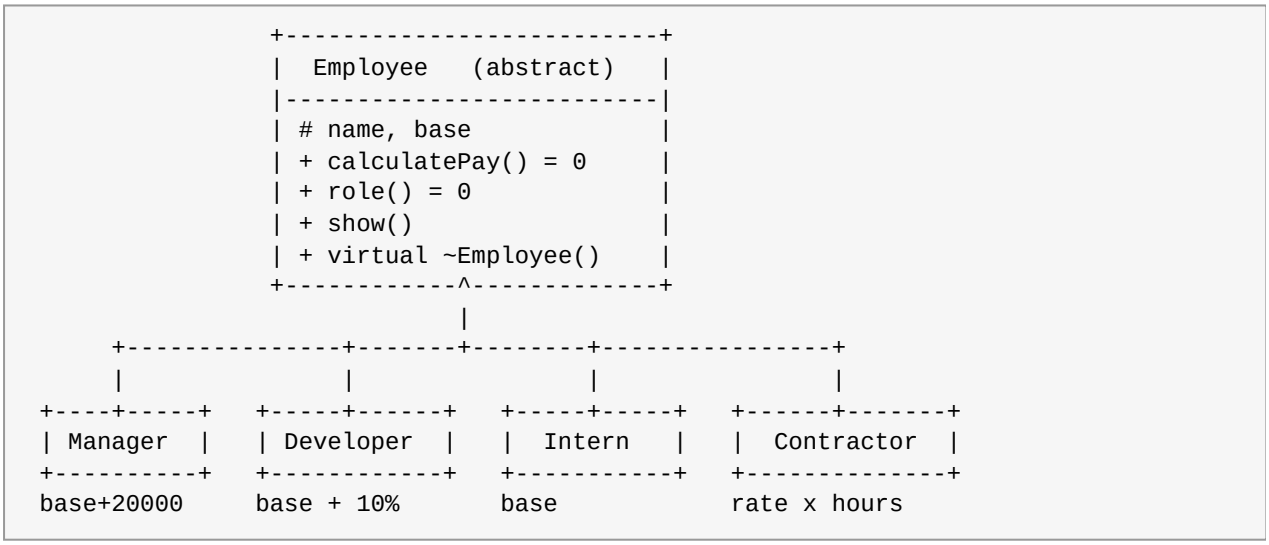
4.4 Pure virtual functions and abstract classes

A virtual function can be declared with `= 0`, which makes it a **pure virtual function**. It has no body in the base class and states only that every derived class must supply one. A class with at least one pure virtual function is an **abstract class**: no object of it can be created, but pointers and references to it are allowed. In the payroll program, `Employee` is abstract because "an employee of no particular type" has no meaningful pay rule.

5. The Solution: Redesign Using Polymorphism

5.1 Design

The redesign turns each employee type into a class that knows its own pay rule. A common abstract base class `Employee` defines the interface that all types share: `calculatePay()` and `role()`.



5.2 Implementation

```
class Employee {
protected:
    string name;
    double base;
```

```
public:
    Employee(string n, double b) : name(n), base(b) {}
    virtual double calculatePay() const = 0;   // pure virtual
    virtual string role() const = 0;
    void show() const {
        cout << role() << " " << name << " : " << calculatePay() << endl;
    }
    virtual ~Employee() {}
};

class Manager : public Employee {
public:
    Manager(string n, double b) : Employee(n, b) {}
    double calculatePay() const override { return base + 20000; }
    string role() const override { return "Manager"; }
};

class Developer : public Employee {
public:
    Developer(string n, double b) : Employee(n, b) {}
    double calculatePay() const override { return base + 0.10 * base; }
    string role() const override { return "Developer"; }
};

class Intern : public Employee {
public:
    Intern(string n, double b) : Employee(n, b) {}
    double calculatePay() const override { return base; }
    string role() const override { return "Intern"; }
};
```

5.3 Adding the new type later

When contractors are introduced, only one new class is written. The classes above and the main loop below stay exactly as they are.

```
class Contractor : public Employee {
    int hours;
public:
    Contractor(string n, double rate, int h) : Employee(n, rate), hours(h) {}
    double calculatePay() const override { return base * hours; }
    string role() const override { return "Contractor"; }
};
```

5.4 The main program

```
int main() {
    Employee* staff[4];
    staff[0] = new Manager("Asha", 60000);
    staff[1] = new Developer("Ravi", 50000);
    staff[2] = new Intern("Meera", 15000);
    staff[3] = new Contractor("Kabir", 500, 80);

    double total = 0;
    for (int i = 0; i < 4; i++) {
        staff[i]->show();
        total += staff[i]->calculatePay();
    }
    cout << "Total payroll : " << total << endl;
```

```
    for (int i = 0; i < 4; i++) delete staff[i];
    return 0;
}
```

The loop has no `if`, no `switch` and no type check. It simply asks each object for its pay, and each object answers according to its own class. Running the program gives:

```
Manager Asha : 80000
Developer Ravi : 55000
Intern Meera : 15000
Contractor Kabir : 40000
Total payroll : 190000
```

The total is correct: 80,000 + 55,000 + 15,000 + 40,000 = 190,000. Kabir's pay is the hourly rate of 500 multiplied by 80 hours.

5.5 Reading the code carefully

- `Employee* staff[4]` is an array of base class pointers. Each element can point to a different derived type, which is how one loop handles all four.
- `calculatePay()` is called through the pointer, so the version that runs is decided at run time by the real object.
- `override` after each function tells the compiler that the function is meant to replace a base class virtual function. If the name or signature is mistyped, the compiler reports an error instead of silently creating an unrelated function.
- `show()` is not virtual. It is written once in the base class and still produces different output for each type, because the two virtual functions it calls are resolved at run time.
- `virtual ~Employee()` is explained in the next section because it prevents a serious bug.

6. Pitfalls and Good Practices

Polymorphism has a few well-known traps. The program below demonstrates two of them, and its output was produced by running it.

```
class Base {
public:
    ~Base() { cout << "Base destroyed" << endl; }       // not virtual
};
class Derived : public Base {
    int* data;
public:
    Derived() { data = new int[100]; }
    ~Derived() { delete[] data; cout << "Derived destroyed" << endl; }
};

class Animal {
public:
    virtual void speak() const { cout << "Animal sound" << endl; }
};
class Dog : public Animal {
public:
    void speak() const override { cout << "Woof" << endl; }
};
```

```
void byValue(Animal a)            { a.speak(); }
void byReference(const Animal& a) { a.speak(); }
```

When `main()` runs `Base* p = new Derived(); delete p;` and then calls `byValue(d)` and `byReference(d)` for a `Dog` object, the output is:

```
Base destroyed
---
Animal sound
Woof
```

6.1 Make the base class destructor virtual

The first line of output is the problem. Only "Base destroyed" appears; "Derived destroyed" never does. Because `~Base()` is not virtual, deleting through a `Base*` runs only the base destructor, so the derived part is never cleaned up and the 100 integers allocated in `Derived` leak. In the payroll program this is why `Employee` declares `virtual ~Employee()`. **Rule:** any class meant to be used through a base pointer must have a virtual destructor.

6.2 Avoid object slicing

`byValue(d)` printed "Animal sound" even though `d` is a `Dog`. Passing a derived object by value to a function that takes the base type copies only the base part and discards the rest, an effect called *slicing*, and the copy is a plain `Animal`. `byReference(d)` printed "Woof" because a reference refers to the original object. **Rule:** pass polymorphic objects by pointer or reference, never by value.

6.3 Other practices

- Always write `override` on overriding functions so mistakes are caught at compile time.
- Use `final` on a class or function when no further overriding should be allowed.
- Prefer pure virtual functions to define an interface that all derived classes must implement.
- Do not make functions virtual without a reason, since each virtual call costs one extra lookup and each object carries a vptr.
- In modern C++, store polymorphic objects in smart pointers such as `unique_ptr<Employee>` so that `delete` is not written by hand.

7. Analysis: Before and After

Aspect	Without polymorphism (if-else on type)	With polymorphism (virtual functions)
Adding a new type	Edit the enum, the shared structure and every function that checks the type	Write one new class; no existing code changes
Where the pay rule lives	In one large function, away from the data	Inside the class of that employee type
Risk of forgetting a case	High; the compiler gives no warning	Low; a pure virtual function must be implemented, or the class cannot be instantiated
Main loop	Needs type checks	Has no type checks
Code duplication	The same chain repeated in each function	Behaviour written once per class
Run-time cost	None	One extra lookup per virtual call and one vptr per object
Best suited to	A fixed and tiny set of types	A set of related types that will change

The comparison shows a clear trade. The price of polymorphism is a very small run-time cost and a little more code at the start. The benefit is that the program can grow safely. The principle behind this is the **open-closed principle**: software should be open for extension (new classes can be added) but closed for modification (existing code is left alone). The payroll redesign satisfies it, and the first design does not.

8. Real-World Applications

The same idea appears wherever a program must treat different kinds of objects through one common interface.

- **Graphics and games:** a list of `Shape` or `GameObject` pointers where every object draws or updates itself in its own way.
- **Graphical user interfaces:** buttons, text boxes and menus are all `Widget` objects that respond to the same `draw()` and `onClick()` calls.
- **Banking:** savings, current and loan accounts all provide `calculateInterest()` with different rules.
- **Plug-in and driver systems:** an application talks to a base `Device` or `Printer` interface, and each manufacturer supplies a derived implementation.
- **Compilers and interpreters:** each node of a syntax tree is a derived class with its own `evaluate()` method.

9. Conclusion

This case study examined polymorphism through a payroll program that had to cope with different employee types and a new type added later. The design without polymorphism worked at first but broke down as it grew, because every change meant editing existing code and repeating type checks. Redesigning with an abstract base class, virtual functions and the `override` keyword allowed the contractor type to be added by writing a single new class, while the main loop stayed unchanged.

The study also covered the two kinds of polymorphism in C++: compile-time (overloading, operator overloading and templates), decided by the compiler, and run-time (virtual functions), decided through the vtable and vptr while the program runs. The tested pitfalls showed that a base class needs a virtual destructor and that polymorphic objects must not be passed by value.

In summary, polymorphism lets code depend on *what an object can do* instead of *what type it is*. That makes programs shorter, safer to change and easier to extend, which is why it is one of the most important features of C++.

10. References

- Bjarne Stroustrup, *The C++ Programming Language*, 4th edition, Addison-Wesley.
- Bjarne Stroustrup, *A Tour of C++*, Addison-Wesley.
- cppreference.com: Virtual function specifier and Abstract classes.
- C++ Core Guidelines, section on class hierarchies: https://isocpp.github.io/CppCoreGuidelines/
- All programs in this report were compiled with GCC using `g++ -std=c++17 -Wall`.